Failure-Aware Motion Planning Using POMDPs

Jacob Lei
Dept. of Aerospace Engineering Sciences
University of Colorado at Boulder
Boulder CO 80303, USA
Jacob.Lei@colorado.edu

Tatsuyoshi Kurumiya
Dept. of Aerospace Engineering Sciences
University of Colorado at Boulder
Boulder CO 80303, USA
Tatsuyoshi.Kurumiya@colorado.edu

***Abstract*—Traditional motion planning methods often generate trajectories under idealized assumptions of deterministic dynamics and perfect actuation. In practice, real-world robotic systems experience unmodeled disturbances and latent failure modes that alter system dynamics and degrade tracking performance. This project proposes a failure-aware motion planning framework that integrates sampling-based trajectory generation with online-decision making under uncertainty using a Partially Observable Markov Decision Process (POMDP). A wheeled robot with unicycle dynamics operating in SE(2) is considered, where an unobserved failure mode induces a persistent control bias.**

Trajectories are generated using the Open Motion Planning Library (OMPL) [1] under both nominal and failure-aware dynamics, while the POMDP governs when to follow the current plan or trigger replanning. Tracking errors between planned and executed trajectories are used as stochastic observations, providing partial information about the latent failure mode.

Multiple POMDP policies are evaluated and compared in terms of tracking performance, planning frequency, and goal-attainment rates. Simulation results show that policies generated from a formulation approximation can accurately detect execution deviations and selectively replan under alternative dynamics, improving robustness to persistent actuation bias.

I. Introduction

Traditional motion planning techniques generate trajectories from a start to goal state under the assumption of deterministic dynamics and perfect actuation. While these assumptions enable efficient planning, they are often violated in real-world settings due to latent failure modes or unmodeled disturbances that degrade system dynamics. As a result, the robot may deviate significantly from its planned trajectory, leading to obstacle collisions or failure to reach its intended goal.

This motivates the need for planning frameworks that adapt online to changes in system behavior. Failures such as actuator bias or partial loss of control are typically not directly observable, and instead must be inferred from deviations between expected and actual motion. This leads to a partially observable formulation, where the underlying system mode is hidden and must be inferred from noisy observations.

Trajectories are generated using the Stable Sparse-RRT (SST) motion planner, a sampling-based algorithm for kinodynamic planning in which both geometric and dynamic constraints constraints are enforced. SST builds a tree of dynamically feasible trajectories through forward propagation and retains only locally optimal states for computational efficiency. However, SST assumes a fixed dynamics model, and trajectories generated under nominal conditions may become invalid when the true system deviates due to latent failures.

In this project, failure-aware motion planning is modeled as a POMDP, where the robot decides whether to execute a previous trajectory or replan in alternative dynamics. The latent mode represents a constant wheel bias that is stochastically activated and deactivated. Stochastic observations are derived from tracking errors between planned and executed trajectories, providing indirect information about the current mode. Multiple policies are evaluated including QMDP and POMCP over an approximate POMDP formulation that simplifies the observation and transition dynamics, avoiding the need to explicitly replan during policy generation.

II. Background and Related Work

Task and Motion Planning (TAMP) is a widely studied field which combines high-level discrete decision-making with continuous motion planning to generate executable trajectories. Typical failures during execution are caused by unmodeled dynamics, sensor noise, and hardware malfunctions, motivating stochastic formulations as POMDPs. Pan et al. address execution failures in TAMP by modeling stochastic action outcomes and maintaining beliefs over the probability of success for each action [2]. These probabilities are updated from experience using a Bayesian framework and incorporated into an MDP to bias planning toward more reliable actions. However, this approach learns action reliability across repeated executions rather than inferring latent causes of failure during online execution.

Causally-informed POMDP methods enable inference over possible failure modes by accounting for hidden factors that influence system dynamics and observations [3]. These effects are learned offline and used during planning to reduce bias from model mismatch. While more robust, these methods introduce additional computation cost from reasoning over latent structure. Motion planning under uncertain vehicle dynamics has also been formulated directly as a POMDP in prior work [4], where the planner maintains a belief over feasible control states and trades off between exploration of unknown dynamics and task execution. Exploration helps identify the feasible operational envelope but introduces risk, requiring a balance between information gain and safety.

Other approaches focus on safety guarantees under imperfect actuation and perception. Chance-constrained POMDPs

[5] bound the probability of failure and incorporate learned estimates of risk into the planning process. Reachability-based methods instead generate trajectories that remain safe under worst-case disturbances [6]. While effective for ensuring safety, these approaches can be overly conservative or computationally expensive for online replanning, and do not explicitly reason about when failures occur during execution.

POMDP-based methods have also been applied to wheeled robot navigation under uncertainty. Deshpande et al. formulate wheeled robot path planning as a belief-space decision problem [7], modeling uncertainty in sensing, actual, and multi-agent interaction. However, this approach relies on a discrete state space and does not address model mismatch during execution.

This work focuses on failure-aware motion planning in continuous, kinodynamic settings, where trajectories are generated using SST and deviations arise from stochastic switching of a latent mode that toggles a constant wheel bias. Solving the full POMDP is intractable due to the continuous state space and the high computational cost of replanning.

An approximate POMDP is constructed that preserves the same action and observation structure as the full model, while replacing complex error propagation with discrete thresholds to approximate tracking error. Policies are computed using traditional POMDP methods such as QMDP [8] and POMCP [9]. These policies are then applied to the full POMDP, enabling efficient online decision-making that uses partially observed tracking error to detect model mismatch and trigger replanning without repeatedly solving the full continuous problem.

III. Problem Formulation

A. Dynamics Formulation & Motion Planner Implementation

SST is chosen as the motion planner for this problem as it converges toward an ideal solution as planning time increases. This ensures consistent and reliable path generation, which is important to achieve stable rewards during POMDP rollout.

The workspace $\mathcal{X} \in \mathbb{R}^2$ is the environment in which the agent operates and is bounded by $x \in [0, 10], y \in [0, 10]$, where the agent is modeled as a point. Within this region, a set of obstacles $\mathcal{X}_{obs}$ is defined. The agent is constrained to movement only within the obstacle-free space where $\mathcal{X}_{free} = \mathcal{X} \setminus \mathcal{X}_{obs}$.

Additionally, the agent's movement is governed by first-order unicycle dynamics (Equation 1) as a simple example of a wheeled vehicle. Therefore, its configuration space $\mathcal{C}$, which represents all parameters needed to define the agent's dynamics within the workspace, is defined as (x, y, θ), where $(x, y) \in \mathcal{X}$ denotes the agent's position and θ represents the heading angle relative to the horizontal axis.

$$\begin{aligned} \dot{x} &= u_{\dot{\phi}} r \cos(\theta) \\ \dot{y} &= u_{\dot{\phi}} r \sin(\theta) \\ \dot{\theta} &= u_{\dot{\theta}} + \omega_b \end{aligned} \tag{1}$$

For this model, a constant wheel radius of $r = 0.5\,\text{m}$ is assumed. The control inputs are defined as $u_{\dot{\phi}}$, the pedaling velocity in rad/s, and $u_{\dot{\theta}}$, the steering rate in rad/s. Finally, a constant yaw rate bias ω_b is incorporated into the angular velocity to simulate an actuator bias, such as a stuck wheel or partial loss of steering control. The value of ω_b is determined by a dynamics mode $m \in \mathcal{M} = \{:\texttt{healthy}, :\texttt{turn\_bias}\}$, where $m = :\texttt{healthy}$ corresponds to $\omega_b = 0$ and $m = :\texttt{turn\_bias}$ corresponds to $\omega_b = 0.8\,\frac{\text{rad}}{\text{s}}$.

When SST is called, the agent begins at an initial configuration $c_{start} \in \mathcal{C}$, such that (x_{start}, y_{start}) lies within the obstacle-free space $\mathcal{X}_{free}$. The objective is defined by a static goal region centered at $c_{goal} \in \mathcal{C}$. A successful trajectory must terminate within a positional tolerance of $\epsilon_s = 0.5$ m from (x_{goal}, y_{goal}) and a heading angle tolerance of $\epsilon_\theta = \frac{\pi}{12}$ rad. While c_{goal} remains fixed for each environment, c_{start} is updated based on the agent's current configuration at the time of the call.

Then, SST solves for a valid trajectory according to the dynamics in Equation 1 for a specific mode $p \in \mathcal{M}$. The solver returns the control sequence $\mathbf{u} = [u_1, \ldots, u_N]$ and the resulting configuration path $\mathbf{x} = [c_{start}, c_1 \ldots, c_N]$, generated by propagating the dynamics using a Runge-Kutta integrator over a timestep $\Delta t = 0.1\,\text{s}$.

Figures 1 and 2 show the main workspace environments that the primary POMDP was evaluated on. Environment 1 features a cluttered workspace, whereas Environment 2 tests performance through a 1-meter-wide corridor.

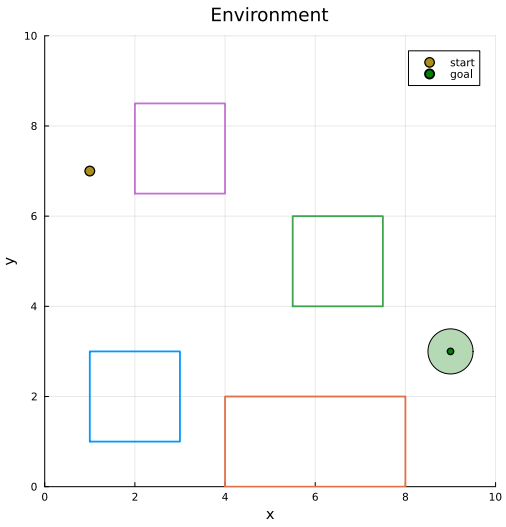

Fig. 1. Environment 1 Configuration

B. Primary POMDP

To plan under actuation failures that introduce an unobserved dynamics bias, the problem is formulated as a POMDP. At each time step, the agent must decide whether to replan a new trajectory using a different dynamics mode $p \in \mathcal{M}$ or commit to an existing trajectory.

The POMDP is defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{O}, T, Z, R, \gamma)$ as follows:

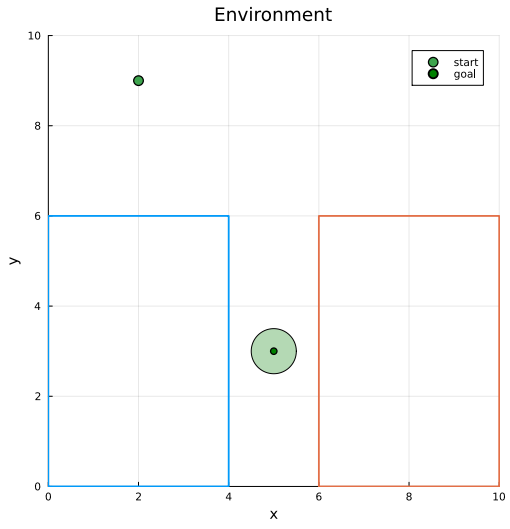

Fig. 2. Environment 2 Configuration

State Space: The state space is defined as follows:

$$s = (\mathbf{x}, m, p, \mathbf{u}, \mathbf{x}_{\text{plan}}, k), \tag{2}$$

where:

- $\mathbf{x} = (x, y, \theta) \in \mathcal{C}$ is the current agent configuration.
- $m \in \mathcal{M}$ is the agent's true dynamics mode, governing ω_b in Equation 1.
- $p \in \mathcal{M}$ is the dynamics mode assumed by the planner in the most recent SST call.
- $\mathbf{u} = \{u_1, \ldots, u_N\}$ is the sequence of control inputs last generated by the planner.
- $\mathbf{x}^{\text{plan}} = \{c_{start}, c_1, \ldots, c_N\}$ is the planned configuration path resulting from applying $\mathbf{u}$ under dynamics mode p with $\Delta t = 0.1$ s.
- $k \in \mathbb{Z}^+$ is the current index within the planned trajectory.

Therefore, when $m \neq p$, $\mathbf{x}$ will diverge from $\mathbf{x}^{\text{plan}}$, necessitating replanning in order to maintain progress towards the goal while avoiding obstacle collision.

Action Space & Transition Model: The action space consists of executing the stored motion plan or triggering a replan under a different planner dynamics mode $p \in \mathcal{M}$:

$$\mathcal{A} = \left\{ \begin{array}{l} \texttt{:continue\_plan}, \\ \texttt{:replan\_nominal}, \\ \texttt{:replan\_failure} \end{array} \right\} \tag{3}$$

where the transitions corresponding to each action are:

- `:continue_plan`: Uses Runge-Kutta numerical integration of Equation 1 to propagate $\mathbf{x} \to \mathbf{x}'$ over timestep $\Delta t = 0.1\,\text{s}$. The integration uses control input $\mathbf{u}[k]$ and true dynamics mode m. Additionally, the dynamics mode m has probability $p_f = 0.01$ to switch between `:healthy` and `:wheel_bias`. Otherwise, $m' = m$. The resultant state is:

$$s' = (\mathbf{x}', m', p, \mathbf{u}, \mathbf{x}_{\text{plan}}, k+1) \tag{4}$$

- `:replan_nominal`: Calls SST to generate a new trajectory $(\mathbf{x}'_{\text{plan}}, \mathbf{u}')$ with $c_{start} = \mathbf{x}$ and mode $p =$ `:healthy`. The resultant state is:

$$s' = (\mathbf{x}, m, \texttt{:healthy}, \mathbf{u}', \mathbf{x}'_{\text{plan}}, 1) \tag{5}$$

- `:replan_failure`: Calls SST to generate a new trajectory $(\mathbf{x}'_{\text{plan}}, \mathbf{u}')$ with $c_{start} = \mathbf{x}$ and mode $p =$ `:turn_bias`. The resultant state is:

$$s' = (\mathbf{x}, m, \texttt{:turn\_bias}, \mathbf{u}', \mathbf{x}'_{\text{plan}}, 1) \tag{6}$$

Observation Space & Function: The observation $o \in \mathcal{O}$ provides a representation of the system's current tracking performance and the planner dynamics mode p:

$$o = (z, p), \tag{7}$$

where:

- $z \in \mathcal{Z}$ is the discretized tracking error level, where $\mathcal{Z} = \{\texttt{:small\_err}, \texttt{:medium\_err}, \texttt{:large\_err}\}$. This value is derived from the true tracking error $\mathbf{e} = \mathbf{x} - \mathbf{x}_{\text{plan}}[k]$, where $e_p = \|\mathbf{e}_{1:2}\|$ represents positional error and $e_\theta = |\mathbf{e}_3|$ represents angular error.
  The tracking error is first classified into three discrete categories according to Equation 8:

$$z_{true} = \begin{cases} \texttt{:small\_error}, & e_p \leq \epsilon_p^s \text{ and } e_\theta \leq \epsilon_\theta^s, \\ \texttt{:medium\_error}, & e_p \leq \epsilon_p^m \text{ and } e_\theta \leq \epsilon_\theta^m, \\ \texttt{:large\_error}, & \text{otherwise.} \end{cases} \tag{8}$$

  In Equation 8, the error thresholds are heuristics, defined $e_p^s = 0.25, e_\theta^s = \frac{\pi}{30}, e_p^m = 0.75, e_\theta^m = \frac{\pi}{6}$.
  Since the observed error reported to the agent is subject to sensing uncertainty, the reported error category z is modeled as a stochastic perturbation of z_{true}. Specifically, the observation model introduces noise such that there is:
  - A 10% probability of reporting a category one step removed from the truth (e.g., `:small_error` reported as `:medium_error`).
  - A 5% probability of reporting a category two steps removed (e.g., `:small_error` reported as `:large_error`).

  This ensures that the policy must remain robust to misclassified error states, mirroring the reliability constraints of real-world sensors.
- p is deterministically observed from the state. This provides the policy with information about the mode assumed by the active trajectory plan.

Terminal States: The terminal states of the POMDP are the collision and goal states. Collision states are defined as states where $\mathbf{x}_{1:2} = (x, y) \in \mathcal{X}_{obs}$, where $\mathcal{X}_{obs}$ is the region of the workspace containing obstacles. The goal region $\mathcal{C}_{goal} \subset \mathcal{C}$ is the region where $\mathbf{x}$ is within a positional tolerance of $\epsilon_s = 0.5\,\text{m}$ and a heading angle tolerance of $\epsilon_\theta = \frac{\pi}{12}$ rad with respect to the goal configuration c_{goal}.

Reward: The reward function is defined to penalize tracking error, discourage unnecessary replanning, and enforce task completion. Therefore, a large negative reward is assigned for s' that are in collision, and a positive reward is assigned for s' within the goal region. For all other s', the cost is based on the tracking error between $\mathbf{x}$ and $\mathbf{x}_{\text{plan}}[k]$. Specifically, Equation 9 details the tracking error function as a function of e_p and e_θ, where $\alpha = 0.5$ is heuristically set.

$$R_{\text{e}} = e_p + \alpha e_\theta \tag{9}$$

Therefore, $R(s')$ is defined as:

$$R(s') = \begin{cases} R_{\text{collision}} = -100, & \mathbf{x}_{1,2} \in \mathcal{X}_{obs} \\ R_{\text{goal}} = 100, & \mathbf{x} \in \mathcal{C}_{goal} \\ R_e = e_p + \alpha e_\theta, & \text{otherwise} \end{cases} \tag{10}$$

Additionally, a moderate negative reward is assigned if the agent decides to replan, which emulates the computational expense of trajectory replanning:

$$R(a) = \begin{cases} -10, & a = \begin{cases} \texttt{:replan\_nominal} \\ \texttt{:replan\_failure} \end{cases} \\ 0, & \text{otherwise} \end{cases} \tag{11}$$

Therefore, the total reward is formulated as:

$$R(s, a, s') = R(s') + R(a) \tag{12}$$

Discount factor: The discount factor is set to $\gamma = 1$ due to the fine time discretization of the system dynamics ($dt = 0.1$), which results in long effective horizons. Using $\gamma < 1$ would excessively discount the terminal rewards, making them negligible relative to accumulated tracking costs. Since episodes terminate upon collision or goal completion, the undiscounted formulation remains well-defined.

C. Baseline POMDP

Before solving the primary POMDP, a baseline version was formulated to evaluate standard policies, such as QMDP and POMCP, on a simplified problem that excludes motion planner integration. In this baseline model, the agent takes actions that predict the dynamics mode $m \in \mathcal{M}$ and receives direct rewards for correct predictions, which reduces the problem to classification.

The baseline POMDP is defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{O}, T, Z, R, \gamma)$ as follows:

State space: The baseline POMDP state space is defined as:

$$s = (m, n_{\text{fail}}) \tag{13}$$

where:

- $m \in \mathcal{M} = \{\texttt{:healthy}, \texttt{:turn\_bias}\}$ is the current dynamics mode of the agent.
- $\{n_{\text{fail}} \in \mathbb{Z} | 1 \leq n_{\text{fail}} \leq 5\}$ is the consecutive number of times that the agent incorrectly predicts m. n_{fail} is used within the reward function to emulate the compounding tracking error cost in the primary POMDP. To maintain a finite state space, n_{fail} is capped at five consecutive fails.

Action space: The actions in the baseline POMDP are:

$$a \in \mathcal{M} \tag{14}$$

where a positive reward is assigned if $a = m$ and a compounding negative reward is assigned if $a \neq m$.

Transition model: Similar to the primary POMDP, m transitions stochastically between : `healthy` and : `turn_bias` with probability $p_f = 0.01$.

n_{fail} transitions according to:

$$n'_{\text{fail}} = \begin{cases} 0, & a = m \\ \min(n_{\text{fails}} + 1, 5), & a \neq m \end{cases} \tag{15}$$

Observation model: The observations $o \in \mathcal{M}$ represent a noisy measurement of the dynamics mode. Crucially, note that o is only a function of the pre-transition state s, since the tracking error in the primary POMDP only gives information about the previous state's dynamics. Formally, when $s = (m, n_{\text{fail}})$:

$$o(s, a, s') = o(s) = \begin{cases} m, & \text{with probability } p_o = 0.8 \\ \neg m, & \text{with probability } 1 - p_o = 0.2 \end{cases} \tag{16}$$

Reward function: The reward function favors actions where $a = m$ and cumulatively penalizes actions where $a \neq m$ according to Equation 17.

$$R(s, a) = \begin{cases} +1, & a = m \\ -2 \cdot n_{\text{fail}}, & a \neq m \end{cases} \tag{17}$$

Discount factor: Since this POMDP has no terminal states, a discount factor of $\gamma = 0.99$ was assigned to ensure the formulation remains well-defined.

This formulation captures failure inference under uncertainty and penalizes mismatch, serving as a surrogate for inferring failure modes and guiding decisions in the full motion planning POMDP.

IV. Solution Approach

A. Baseline POMDP

The baseline POMDP is a discrete state POMDP with stochastic transition and observation functions, allowing it to be solved with most conventional POMDP solvers, such as QMDP and POMCP. However, because the observation function depends on the pre-transition state s rather than the post-transition state s', many off-the-shelf Point-Based Value Iteration (PBVI) solvers, such as SARSOP, cannot be applied to the baseline POMDP.

The baseline POMDP was evaluated on four different policies, where the first two policies are naive policies to provide a performance floor:

1) a = :healthy: This policy always takes the :healthy action, regardless of the observation.
2) a = :turn_bias: This policy always takes the :turn_bias action, regardless of the observation.
3) QMDP: This policy applies a basic QMDP solver, which solves the underlying MDP after the first step. QMDP is expected to achieve high performance in this specific environment because the observation o is independent of the action a. Consequently, QMDP's inability to accurately represent the value of costly information gathering is not an issue.
4) POMCP: Instead of using a traditional rollout function for value estimation in POMCP, this implementation of POMCP uses value iteration on the underlying MDP to approximate the value of each state. While this heuristic provides an optimistic value approximation, this implementation of POMCP is estimated to have high performance since o is independent of a.

B. Primary POMDP

Compared to the baseline POMDP, the primary POMDP is much harder solve for the following reasons:

- The state space is continuous and therefore infinite, precluding the use of discrete solvers.
- The transition and observation probabilities do not have closed form solutions, which complicates Bayesian belief updates and solution methods for the underlying MDP.
- The transition functions for $a = \{$:replan_nominal, :replan_failure$\}$ call the SST motion planner, which has a runtime of five seconds per call. Therefore, exploration-based reinforcement learning solutions such as Q-Learning and POMCP become computationally infeasible.

1) Formulation Approximation: The proposed approach solves a formulation approximation (ApPrOMDP) of the primary POMDP, where the ApPrOMDP has a discrete, finite state space and simplified transition and observation functions. In this setup, observations from the primary POMDP are fed into the ApPrOMDP, which then generates actions to be applied back to the primary system (Figure 3).

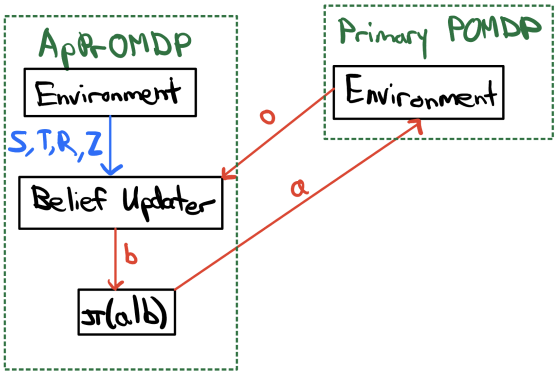

Fig. 3. ApPrOMDP Formulation Approximation Information Flow

While the ApPrOMDP shares the same action space $\mathcal{A}$ and discrete observation space $\mathcal{O}$ as the primary POMDP, it utilizes distinct state spaces, transitions, rewards, and observation functions. A well-constructed approximation ensures that a policy optimized for the ApPrOMDP maintains high performance when deployed on the primary POMDP.

The ApPrOMDP uses two integer hyperparameters, $n_{\text{wrong,med}} = 2$ and $n_{\text{wrong,large}} = 5$, to approximate error dynamics. These parameters categorize the cumulative instances where $m \neq p$ before a "medium" or "large" error state is officially triggered within the approximation. Since the ApPrOMDP utilizes action space $\mathcal{A}$ and observation space $\mathcal{O}$ from the primary POMDP, only $\mathcal{S}_a, \mathcal{T}_a, Z_a, R_a, \gamma_a$ of the ApPrOMDP formulation are outlined here.

State Space & Transition Model:

$$s = (m, p, n_{\text{wrong}}) \in \mathcal{S}_a \tag{18}$$

where:

- $m \in \mathcal{M}$ represents the current dynamics mode, as in the primary POMDP. When $a =$:continue_plan, m has probability $p_f = 0.01$ to transition to $m' = \neg m$. Otherwise, $m' = m$.
- $p \in \mathcal{M}$ represents the planner dynamics mode, as in the primary POMDP. When $a =$:replan_nominal, $p' =$:healthy and when $a =$:replan_failure, $p' =$:turn_bias
- $\{n_{\text{wrong}} \in \mathbb{Z} | n_{\text{wrong}} < n_{\text{wrong, large}}\}$ captures the number of consecutive times where $m \neq p$, resetting to zero if $m = p$.

Observation Function, Reward, & Discount

The observation function utilizes simplified error dynamics, where the true error z_{true} is:

$$z_{true} = \begin{cases} \text{:small\_error}, & n_{\text{wrong}} < n_{\text{wrong,med}}, \\ \text{:medium\_error}, & n_{\text{wrong}} < n_{\text{wrong,large}}, \\ \text{:large\_error}, & \text{otherwise.} \end{cases} \tag{19}$$

Noise is introduced to z_{true} using the same model as the primary POMDP to produce observation component z. p is obtained deterministically from the state, consistent with the primary POMDP.

The reward function is an optimistic variant of the primary POMDP reward, where $R(s')$ (Equation 10) is replaced with Equation 20, which assumes the tracking error is the minimum for a given z_{true} category. Note that $\epsilon_p^s = 0.25, \epsilon_\theta^s = \frac{\pi}{30}, \epsilon_p^m = 0.75, \epsilon_\theta^m = \frac{\pi}{6}$ are defined in the error threshold formulation for the primary POMDP and $\alpha = 0.5$ is defined in the reward section.

$$R(s') = \begin{cases} 0, & n_{\text{wrong}} < n_{\text{wrong,med}}, \\ \epsilon_p^s + \alpha\epsilon_\theta^s, & n_{\text{wrong}} < n_{\text{wrong,large}}, \\ \epsilon_p^m + \alpha\epsilon_\theta^m, & \text{otherwise.} \end{cases} \tag{20}$$

Since there is no way to approximate collisions or the goal region without calling the motion planner, the terminal

rewards and states are omitted in this formulation, resulting in an infinite horizon POMDP. Therefore, a discount factor of $\gamma = 0.99$ is applied to ensure the problem is well-posed.

2) Policies: The primary POMDP was evaluated against three distinct policies, beginning with a naive policy designed to establish a performance floor:

1) Continue: This policy always takes the action $a =$ `:continue_plan`, regardless of the observation.
2) QMDP: This policy applies a standard QMDP solver to the ApPrOMDP. It generates beliefs and actions by filtering observations from the primary POMDP through the transition dynamics of the ApPrOMDP (Figure 3). Since the observation dynamics contain the biggest approximation in the ApPrOMDP formulation, QMDP is expected to achieve relatively high performance because it ignores observation dynamics after the first step and QMDP's inability to represent the value of costly information gathering is not an issue.
3) POMCP: Similar to the QMDP policy, this policy applies a standard POMCP solver to the ApPrOMDP, maintaining an online belief representation by simulating trajectories through the approximated dynamics. As with the baseline POMDP, the leaf values are approximated using value iteration. Unlike QMDP, which assumes full observability after the initial step, POMCP accounts for future uncertainty through Monte Carlo Tree Searches. However, since the observation dynamics in ApPrOMDP use hyperparameters to approximate the complex error dynamics, the resulting belief estimates may diverge significantly from the true system state. Consequently, POMCP is expected to underperform relative to QMDP.

V. Baseline POMDP Results

The maximum cumulative reward on the baseline POMDP is achieved when the agent selects the correct action at every time step, yielding a reward of $+1$ per step. For a finite horizon of $T = 500$ with discount factor $\gamma = 0.99$, the maximum return is

$$R_{\max,\gamma} = \sum_{t=0}^{499} \gamma^t = \frac{1-\gamma^{500}}{1-\gamma} \approx 99.34. \tag{21}$$

The results over 1000 Monte Carlo simulations for the four polices on the baseline POMDP are shown in Table I. The fixed policies perform poorly, with strong negative returns ($\mu = -248.11$ for $a =$ `healthy` and $\mu = -598.42$ for $a =$ `turn_bias`). This happens because any mismatch between the assumed and true system mode leads to penalties that accumulate over time. Both QMDP and POMCP achieve significantly higher performance, with positive returns ($\mu = 70.58$ and $\mu = 68.09$, respectively) and low standard error (SEM < 1), indicating stable performance across Monte Carlo runs.

These results show that both QMDP and POMCP correctly adapt their actions based on the inferred system mode, which prevents the accumulating penalties seen in fixed policies. While QMDP slightly outperforms POMCP, the gap is small because both policies use value iteration for value estimation and observations are independent of actions. Overall, the results show that QMDP and POMCP can handle latent failures with noisy observations.

TABLE I
Monte Carlo Evaluation for 1000 Runs

Policy	Mean Return (μ)	SEM
$a =$ `:healthy`	-248.11	8.14
$a =$ `:turn_bias`	-598.42	8.04
QMDP	**70.58**	**0.61**
POMCP	68.09	0.55

VI. Main POMDP Results

A. Naive Policy

The naive policy, which always selects `:continue_plan`, serves as a baseline for executing the original trajectory under unmodeled failures. As shown in Figs. 4–5, when a wheel bias occurs, the executed trajectory diverges significantly from the nominal plan, often resulting in large tracking errors and collision with obstacles. This is reflected in Table II, where the naive policy yields low mean returns ($\mu = -172.79$ and $\mu = -784.48$ for Environments 1 and 2, respectively) with high variance, indicating unreliable performance under failure.

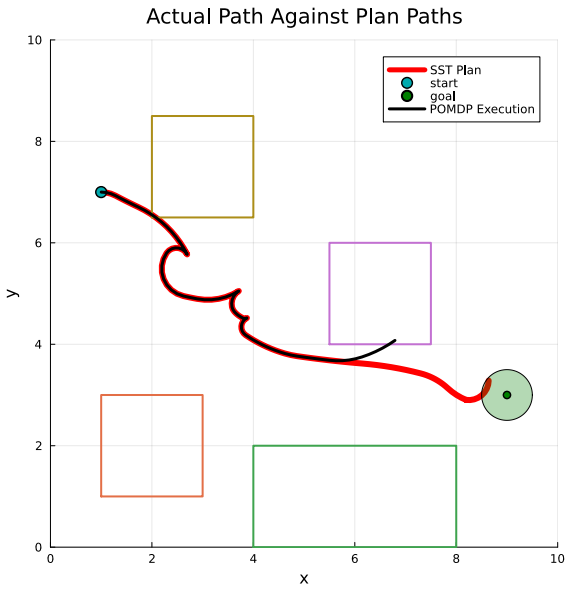

Fig. 4. Environment 1: Trajectories for nominal and POMDP motion planner under naive policy.

B. Approximate POMDP Solutions

1) QMDP Solution on ApPrOMDP: The QMDP policy computed on the ApPrOMDP performs significantly better than both the naive policy and POMCP when evaluated on the full POMDP. As shown in Table II, QMDP achieves positive mean returns in Environment 1 ($\mu = 44.27$) and Environment 2 ($\mu = 49.36$). The policy successfully detects deviations from

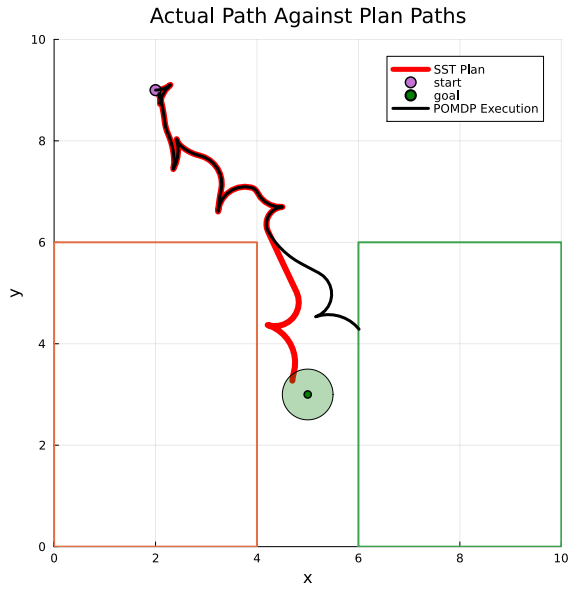

Fig. 5. Environment 2: Trajectories for nominal and POMDP motion planner under naive policy.

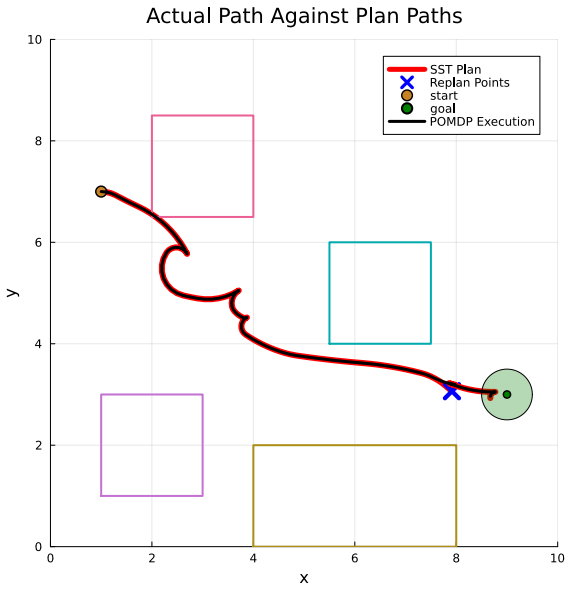

Fig. 6. Environment 1: Trajectories for nominal and POMDP motion planner under QMDP policy.

tracking error observations and triggers replanning under the correct dynamics mode ($p = m$), enabling the robot to reach the goal (Figs. 6–7).

This matches the expectation of a good policy, where performance depends on detecting dynamics mismatch and replanning at the correct time. Since the ApPrOMDP provides an approximation for accumulated tracking error, QMDP can derive an α-vector policy that switches dynamics modes once belief that $m \neq p$ becomes significant. Observations are driven by tracking error and are independent of the chosen action, so there are no information gathering actions, making QMDP well-suited to this problem.

2) POMCP Solution on ApPrOMDP: The POMCP policy solved on the ApProMDP performs better than the naive policy but worse than QMDP when evaluated on the full POMDP. As shown in Table II, POMCP yields negative mean returns ($\mu = -67.44$ and $\mu = -281.98$) with high variance. In general, POMCP and QMDP return similar trajectories but POMCP tends to allow more error to accumulate before replanning. Unlike QMDP, POMCP evaluates actions using simulated rollouts in the approximate model. The ApPrOMDP only provides a course representation of the true observation dynamics, so the belief estimate from the tree search does not accurately reflect how tracking error evolves in the continuous system. As a result, there is limited benefit to planning over belief.

This causes POMCP to overfit to the approximate POMDP and misjudge when to delay or trigger replanning, since tracking error accumulates differently in the continuous system. The resulting policy leads to suboptimal decisions and increased variance when deployed on the full POMDP. QMDP avoids this issue by ignoring belief after the first action, which better matches the structure of the problem.

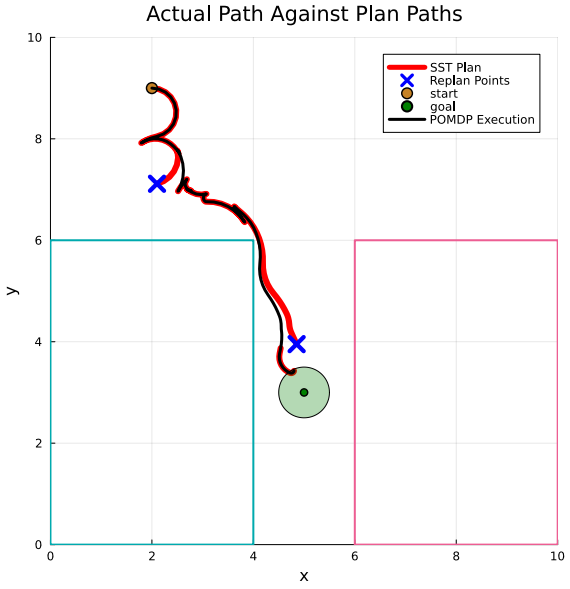

Fig. 7. Environment 2: Trajectories for nominal and POMDP motion planner under QMDP policy.

C. Summary of results

Overall, these results show that effective performance in the main POMDP depends primarily on successfully detecting when the assumed dynamics diverge from the true system and responding with timely replanning. The approximate POMDP formulation successfully captures this structure through the simplified error dynamics, enabling simple policies such as QMDP to perform well when transferred to the full system. POMCP does not perform well in this setting due to overfitting to the approximate model.

TABLE II
MONTE CARLO EVALUATION FOR 100 RUNS ACROSS ENVIRONMENTS

Policy	μ (Env 1)	SEM (Env 1)	μ (Env 2)	SEM (Env 2)
Continue	-172.79	46.24	-784.48	166.56
QMDP	**44.27**	**10.51**	**49.36**	**11.08**
POMCP	-67.44	21.79	-281.98	118.32

VII. CONCLUSION

This work studies failure-aware motion planning under hidden dynamics mismatch, where an intermittent wheel bias causes deviation from planned trajectories. The key challenge is detecting when the assumed dynamics no longer match the true system and triggering replanning at the right time, rather than long-horizon belief optimization. Because directly solving the full belief-space POMDP is computationally intractable, an approximate POMDP was introduced, replacing continuous dynamics with a simplified model that captures accumulated tracking error.

Results demonstrate that policies computed on this approximation can be effectively deployed on the full POMDP to avoid the computational cost associated with running SST. In particular, the QMDP policy achieves strong performance because neither the main POMDP nor the ApPrOMDP have information-gathering actions, resulting in good value approximations relative to other states. POMCP exhibits degraded performance due to its reliance on belief estimates from tree expansion. Since the ApPrOMDP is an approximation of the primary POMDP, the overfitting of belief estimates to the ApPrOMDP results in suboptimal actions from POMCP. This highlights that more general POMDP solvers, such as POMCP, do not necessarily perform better in approximate formulations, especially when the problem structure does not include active information gathering.

Future work may extend this framework to handle multiple failure modes, richer observation models, and adaptive approximations that better capture continuous system behavior while maintaining computational tractability. The POMDP can be extended to include an information gathering `:wait` action, remaining sedentary with an associated negative reward.

VIII. CONTRIBUTIONS AND RELEASE

Jacob:

1) Implemented all POMDP solutions and Monte Carlo simulations for the baseline POMDP
2) Implemented the ApPrOMDP and its corresponding QMDP and POMCP solutions.
3) Created the library to interface OMPL (in C++) with Julia,
4) Report writing (Problem Formulation & Solution Approach)

Tatsu:

1) Implemented the baseline and main POMDP in Julia
2) Developed the interface between the motion planner and the POMDP environment
3) Attempted a Bayesian belief update approach for the main POMDP, which proved that explicit belief space planning is intractable
4) Report writing (Abstract, Introduction, Related Works, Results, Conclusion)

Custom Implementations vs Off-the-Shelf:

- The SST solver was **off-the-shelf** from the OMPL library for C++. However, the integration of OMPL into a Julia library and into the POMDPs were implemented by us.
- QMDP and the basic POMCP algorithm were **off-the-shelf** from their respective Julia packages.
- The ApPrOMDP problem formulation and all problem formulations were **implemented by us**.
- The naive policies were **implemented by us**.

The authors grant permission for this report to be posted publicly.

REFERENCES

[1] I. A. Sucan, M. Moll, and L. E. Kavraki, "The open motion planning library," *IEEE Robotics Automation Magazine*, vol. 19, no. 4, pp. 72–82, 2012.

[2] T. Pan, A. M. Wells, R. Shome, and L. E. Kavraki, "Failure is an option: Task and motion planning with failing executions," in *2022 International Conference on Robotics and Automation (ICRA)*, 2022, pp. 1947–1953.

[3] R. Cannizzaro and L. Kunze, "Car-despot: Causally-informed online pomdp planning for robots in confounded environments," in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023, pp. 2018–2025.

[4] N. Meuleau, C. Plaunt, D. Smith, and T. Smith, "A pomdp for optimal motion planning with uncertain dynamics," 05 2010.

[5] R. J. Moss, A. Jamgochian, J. Fischer, A. Corso, and M. Kochenderfer, "Chance-constrained POMDP planning with learned neural network surrogates," in *IJCAI 2024 Workshop on Trustworthy Interactive Decision-Making with Foundation Models*, 2025. [Online]. Available: https://openreview.net/forum?id=w5a6C4tiat

[6] Y. Wang, A. A. R. Newaz, J. D. Hernández, S. Chaudhuri, and L. E. Kavraki, "Online partial conditional plan synthesis for pomdps with safe-reachability objectives: Methods and experiments," *IEEE Transactions on Automation Science and Engineering*, vol. 18, no. 3, pp. 932–945, 2021.

[7] S. V. Deshpande, H. R, and R. Walambe, "Pomdp-based probabilistic decision making for path planning in wheeled mobile robot," *Cognitive Robotics*, vol. 4, pp. 104–115, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2667241324000077

[8] M. L. Littman, A. R. Cassandra, and L. P. Kaelbling, "Learning policies for partially observable environments: Scaling up," in *Machine Learning Proceedings 1995*, A. Prieditis and S. Russell, Eds. San Francisco (CA): Morgan Kaufmann, 1995, pp. 362–370. [Online]. Available: https://www.sciencedirect.com/science/article/pii/B9781558603776500529

[9] D. Silver and J. Veness, "Monte-carlo planning in large pomdps," in *Advances in Neural Information Processing Systems*, J. Lafferty, C. Williams, J. Shawe-Taylor, R. Zemel, and A. Culotta, Eds., vol. 23. Curran Associates, Inc., 2010. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2010/file/edfbe1afcf9246bb0d40eb4d8027d90f-Paper.pdf